scheduling

↓

Replacement

Chapter 9: Virtual Memory Management

Chien Chin Chen

Department of Information Management
National Taiwan University

Background (1/3)

- Methods in Chapter 8 require that an **entire** process be in memory before it can execute.
  - The requirement is reasonable, but limits the size of a program to the size of physical memory.
- In many cases, the entire program is not needed!!
  - Programs handling unusual error conditions are rarely executed.
  - Arrays, lists, and tables are often allocated more memory than they actually need.

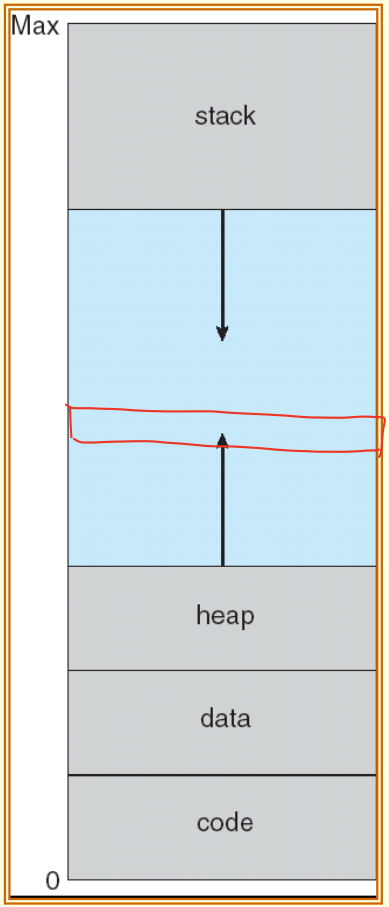
Background (2/3)

- Benefits of executing a program that is **partially** in memory:
  - A program would no longer be constrained by the amount of physical memory that is available.
  - More programs could be run at the same time.
    - Increase CPU utilization and throughput.
  - Less I/O would be needed to load or swap each user program into memory, so user programs would run faster.

- ***Virtual memory*** is a technique that allows the execution of processes that are not completely in memory.

Background (3/3)

- ***Virtual address space***:
  - The **logical** (or virtual) **view** of how a process is stored in memory.
  - Logically, a process begins at a certain logical address – say, address 0 – and exists in contiguous memory.
- Virtual address spaces that include **holes** are known as ***sparse address space***.
  - Is beneficial because the holes can be filled during program execution.
  - For example, the large blank space between the heap and stack.



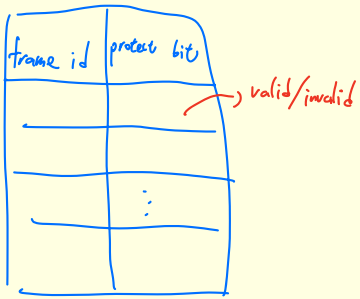
Demand Paging – **Basic Concepts** (1/7)

- ***Demand paging*** is a technique commonly used in virtual memory systems.
- Pages are only loaded when they are demanded during program execution.
  - Pages that are never accessed are thus never loaded into physical memory.
- Initially, a process resides in secondary memory (usually a disk).
- Then we swap it into memory.
  - Rather than swapping the entire process into memory, we use a ***lazy swapper*** (pager). disk上某一個 page 搬到 memory 某一個 frame
  - Lazy swapper never swaps a page into memory unless that page will be needed.
    ↓
    有需要才搬

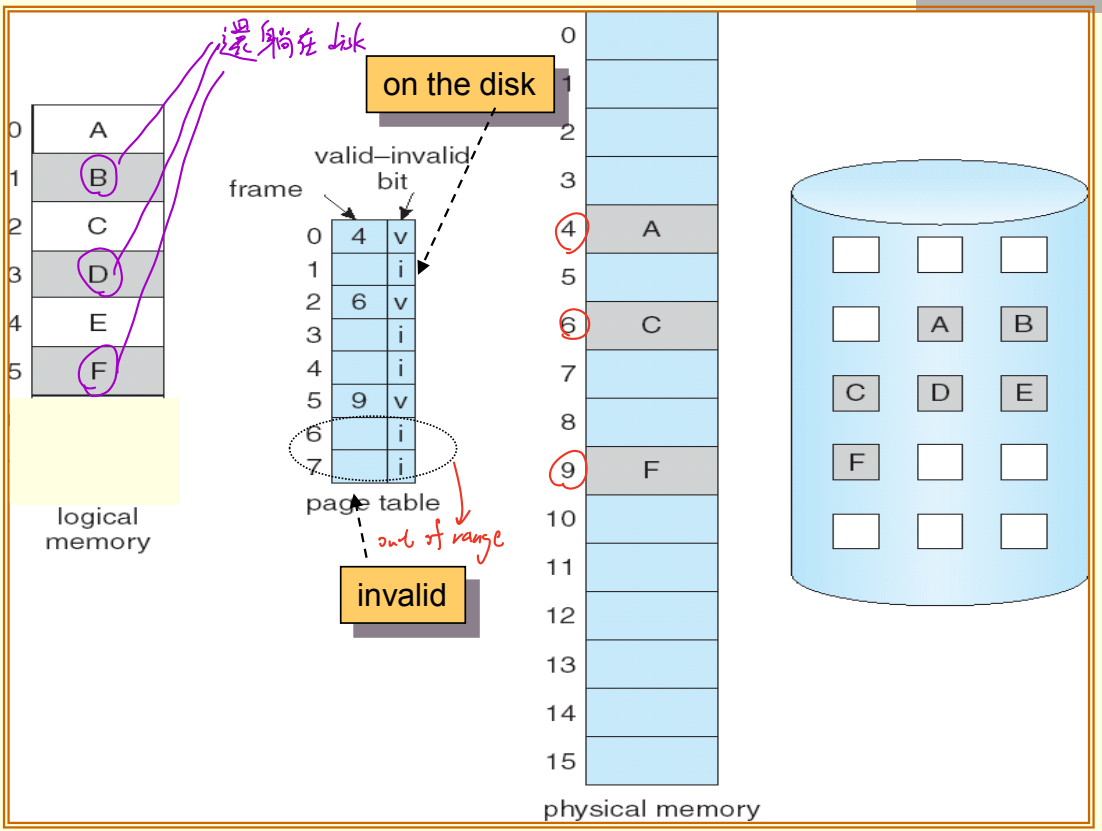
Demand Paging – Basic Concepts (2/7)

- We need hardware support to distinguish between the pages that are in memory and the pages that are on the disk.

- The **valid-invalid bit scheme** (Chapter 8) can be used for this purpose.
  - When valid, the associated page is both legal and in memory.
  - When invalid, the associated page either is not valid or is valid but is currently on the disk.



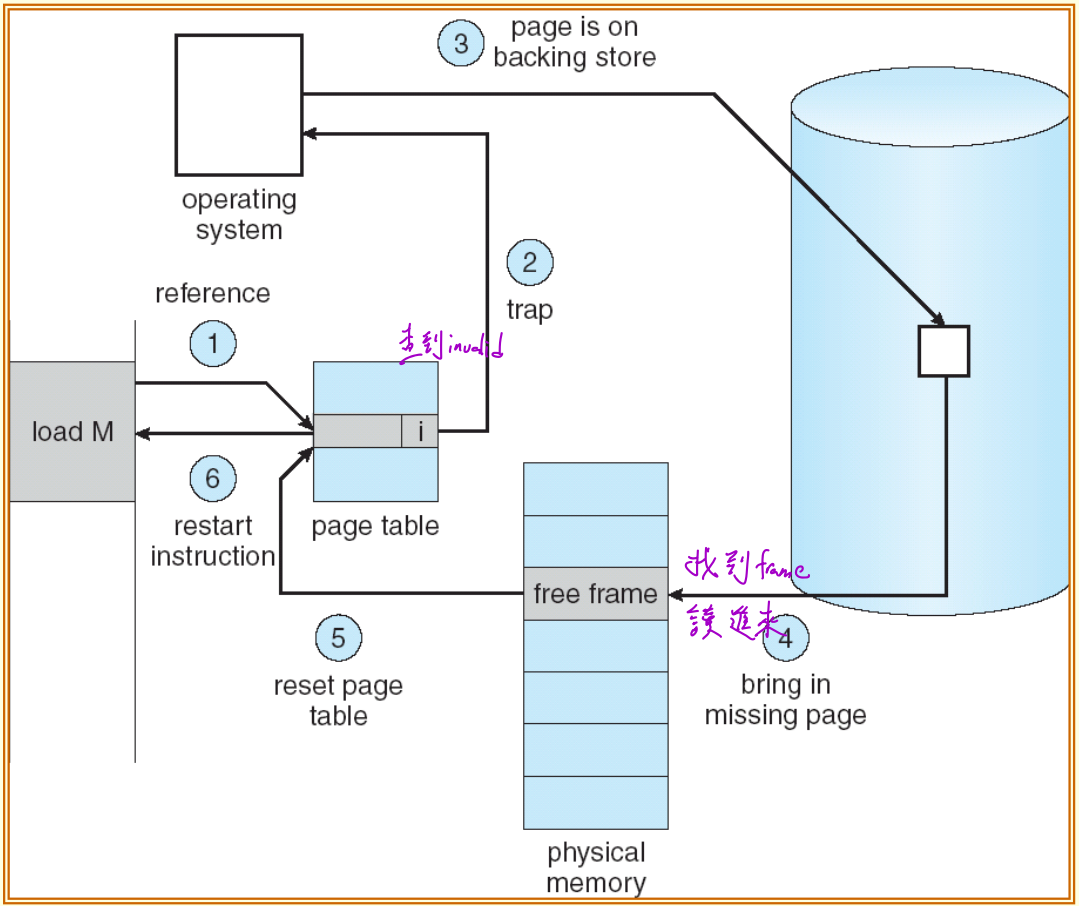
Demand Paging – Basic Concepts (3/7)



Demand Paging – Basic Concepts (4/7)

- Access to a page marked invalid causes a ***page-fault trap***.
- The procedure for handling a page fault:
  1. We check an internal table for this process to determine whether the reference was a valid or an invalid memory access.
  2. If invalid, we terminate the process.
  3. If valid, but we have not yet brought in that page, we now page it in.
  4. We find a free frame. 找到 frame
     - Frame table (Chapter 8).
  5. We schedule a disk operation to read the desired page into the newly allocated frame.
  6. When the disk read is complete, we modify the internal table and page table to indicate that the page is now in memory.
  7. We **re-start** the instruction that was interrupted by the trap.

Demand Paging – Basic Concepts (5/7)



Demand Paging – Basic Concepts (6/7)

- If a page fault occurs on the instruction fetch …
  - We can re-start by fetching the instruction again.
- If a page fault occurs while we are fetching an operand …
  - We must fetch and decode the instruction **again**.
  - And then fetch the operand.
  - Example – `ADD` the content of `A` to `B`, placing the result in `C`.
    1. Fetch and decode the instruction (`ADD`)
    2. Fetch `A`.
    3. Fetch `B`.
    4. Add `A` and `B`.
    5. Store the sum in `C`.

We must repeat steps 1 to 4 after bringing the desire page,

If we fault when storing C.

If you are unlucky, you may have faults in step 1, 2, 3, and 5, respectively!!

Demand Paging – Basic Concepts (7/7)

- ***Pure demand paging scheme***:
  - Execute a process with **no** pages in memory intially.
  - The process immediately faults for the page of the first instruction.
  - After this page is brought into memory, the process continues to execute.
    - Faulting as necessary until every page that it needs is in memory.

Demand Paging – **Performance** (1/3)

- The ***effective access time*** for a demand-paged memory.

  effective access time = (1 – *p*) x *ma* + *p* x *page fault time*

  Probability of a page fault

  Memory-access time

- Three major components of the ***page-fault service time***:
  1. Service the page-fault interrupt.
  2. Read in the page.
  3. Re-start the process.

  - The first and third tasks may take from 1 to 100 microseconds each.
  - The second task will probably be close to 8 milliseconds.
    - 3 milliseconds for an average hard disk latency.
    - 5 milliseconds for a hard disk seek.
    - 0.05 milliseconds for a page transfer.
  - Thus, the total paging time is about 8 milliseconds, including hardware and software time.

Demand Paging – Performance (2/3)

- The page time can be even longer is we have to queue the disk I/O request (when other process are using the disk).
- If we take an average page-fault service time of 8 milliseconds and a memory-access time of 200 nano-seconds, then the effective access time (nanoseconds) is :

$$\textit{effective access time} = (1 - p) \times 200 + p \times 8{,}000{,}000$$
$$= 200 + 7{,}999{,}800 \times p$$

Proportional to page-fault rate.

Demand Paging – Performance (3/3)

- If p = 1/1000, then effective access time is 8.199 microseconds.
  - The computer will be **slowed down by a factor of 40.99 because of demand paging**!!

慢40倍

- If we want degradation to be less than 10 percent, we need

  $220 > 200 + 7{,}999{,}800p$ or $\boldsymbol{p < 0.0000025}$.

  - That is, fewer than one memory access out of 399,990 to page-fault … **that is impossible in a demand-paging system**.

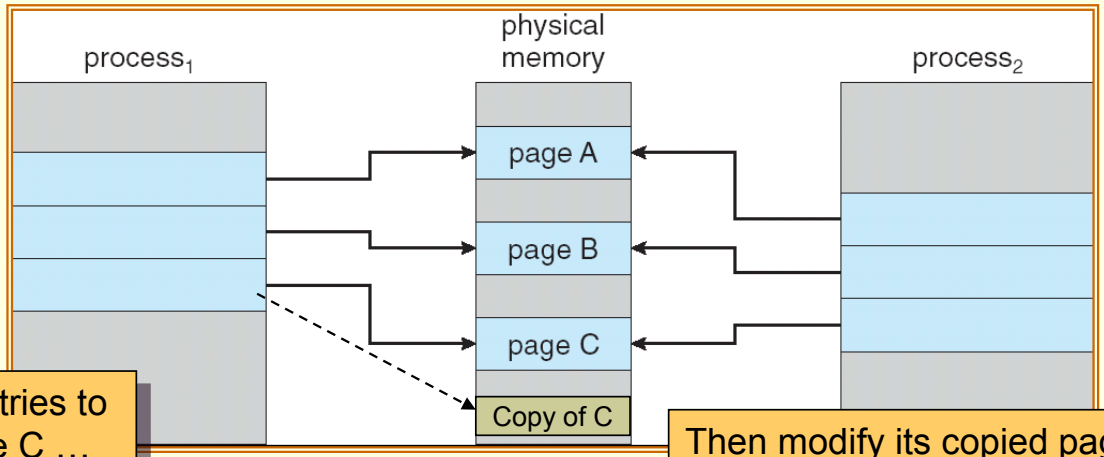
幾乎辦不到

Copy-on-Write (1/2)

- Traditionally, `fork()` worked by creating a copy of parent's address space for the child.
  - Duplicating the pages belonging to the parent.
- Many child processes invoke the `exec()` system call immediately after creation.
  - The copying of the parent's address space may be unnecessary.
- ***Copy-on-write*** is a technique that allows the parent and child processes initially to **share** the same page!!
  - Provide for rapid process creation.
  - Minimize the number of allocated pages.

Copy-on-Write (2/2)

- The shared pages are marked as copy-on-write pages.
  - Meaning that if either process writes to a shared page, a copy of the shared page is created.

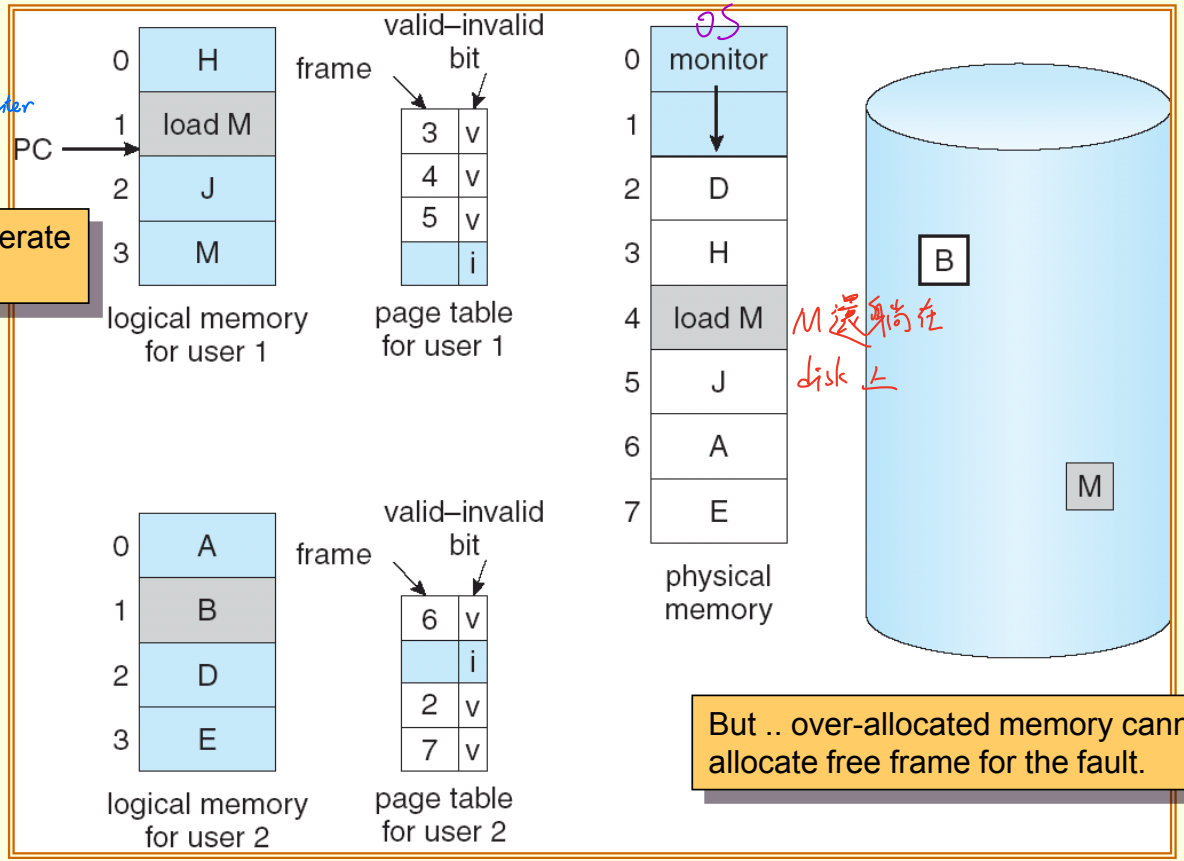


- Copy-on-write is a common technique used by several operating system, including Windows XP, Linux, and Solaris.

Page Replacement – Basic (1/8)

- With demand paging, we can **over allocate memory**!!
  - To increase the degree of multiprogramming.
- Assuming a process of ten pages generally uses half of them.
  - Five pages are rarely used (loaded).
- For a system with forty frames, we could run six processes (rather than four) with ten frames to spare.
- However, it is possible that each of these process may suddenly try to use all ten of its pages.
  - Resulting in a need for sixty frames when only forty are available.

Page Replacement – Basic (2/8)



Access M generate a page fault

But .. over-allocated memory cannot allocate free frame for the fault.

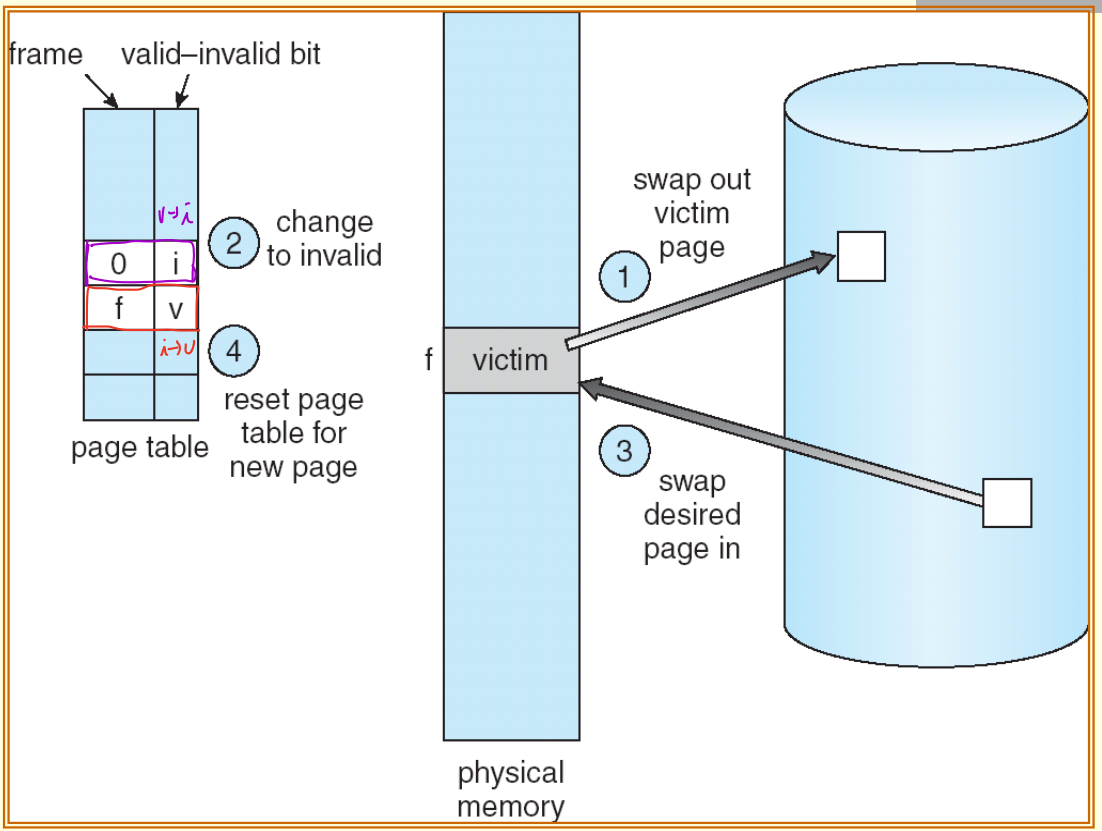
Page Replacement – Basic (3/8)

- To solve the page fault:
  1. The operating system could terminate the process.
     - A very bad idea. Paging should be logically transparent to the user.
  2. The operating system could swap out a process, freeing all its frames.
     - Costly, and reduce the level of multiprogramming.
  1. The most common solution – ***page replacement***.

Page Replacement – Basic (4/8)

- Page-fault service routine including page replacement:
  1. Find the location of the desired page on the disk.
  2. Find a free frame:
     - If there is a free frame, use it. frame 還有 就用
     - If not, use a ***page-replacement algorithm*** to select a victim frame.
     - Write the victim frame to the disk; change the page and frame tables accordingly.
  3. Read the desired page into the newly free frame; change the page and frame tables.
  4. Re-start the user process.

Page Replacement – Basic (5/8)



Page Replacement – Basic (6/8)

- If no frames are free, two page transfer (one out and one in) are required.
  - **Double the page-fault service time!!**
- We can reduce the overhead by using a ***modify bit*** (or dirty bit).
  - The modify bit for a page is set whenever any word or byte in the page is written into, since it was read in from the disk.
- When we select a page for replacement, we examine its modify bit.
  - If set, we must write that page to the disk.
  - If not set, we need not write the page to the disk.

Page Replacement – Basic (7/8)

- There are many page-replacement algorithms. **How do we select a particular replacement algorithm**?
  - In general, we wan the one with the lowest page-fault rate.
- We compute the number of page faults of an algorithm by running it on a string of memory references – ***reference string***.
  - The string can be generated artificially, or by tracing a given system.
- **We only consider the page number**, rather than the specific address.
  - If page *p* is valid, any reference to *p* will never cause a fault.

So the following address sequence:

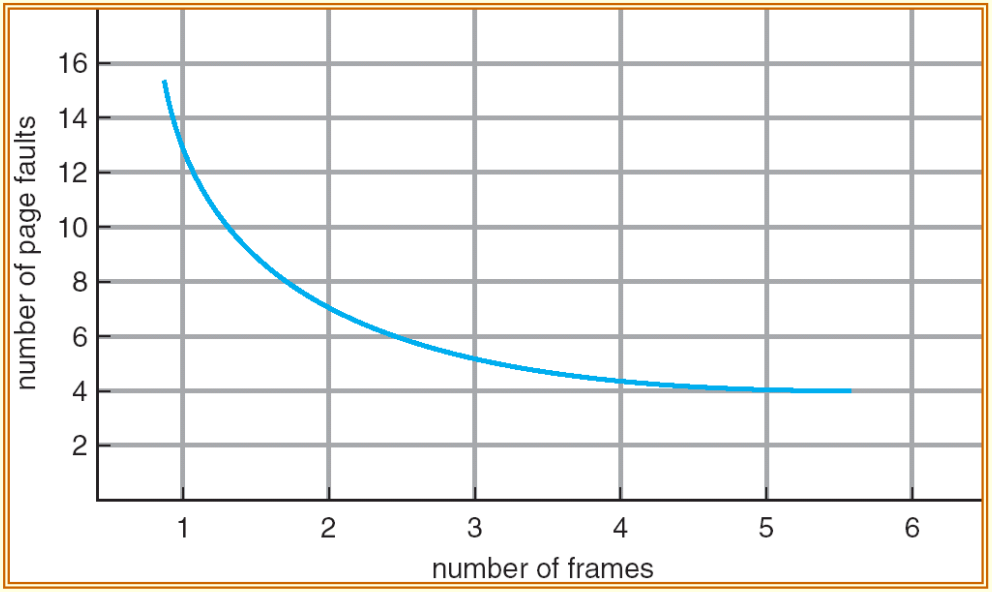
0100, 0432, …, 0609, 0102.

is reduced to the following reference string, at 100 bytes per page:

1, 4, …, 6, 1

Page Replacement – Basic (8/8)

- To calculate the number of page faults, we need to know the number of frames available.
- Ideally, as the number of frames available increases, the number of page faults decreases.
  - However, the assumption is not always true!!



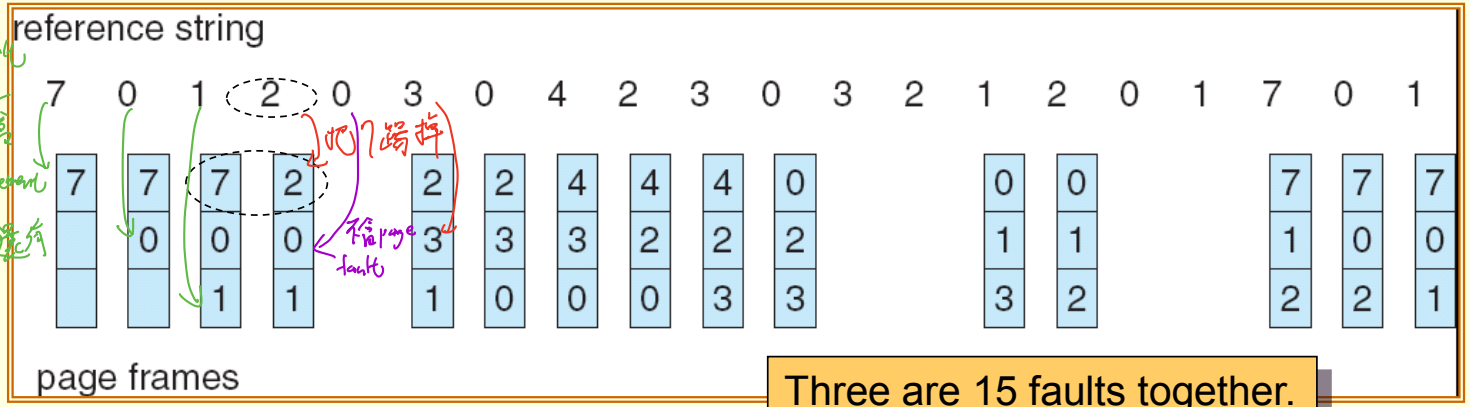
replacement ≠ page fault

FIFO Page Replacement (1/2)

- A ***FIFO replacement algorithm*** (first-in, first-out) associates with each page the time when it was **brought** into memory.
- When a page must be replaced, the oldest page is chosen.

把最老的踢掉

- Example – for a memory with three frames:

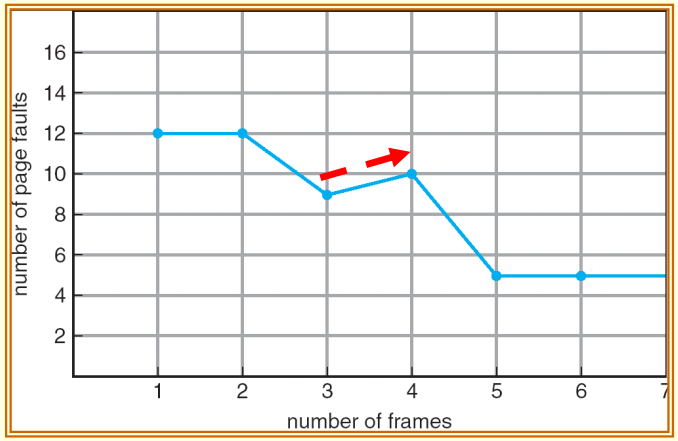


Three are 15 faults together.

FIFO Page Replacement (2/2)

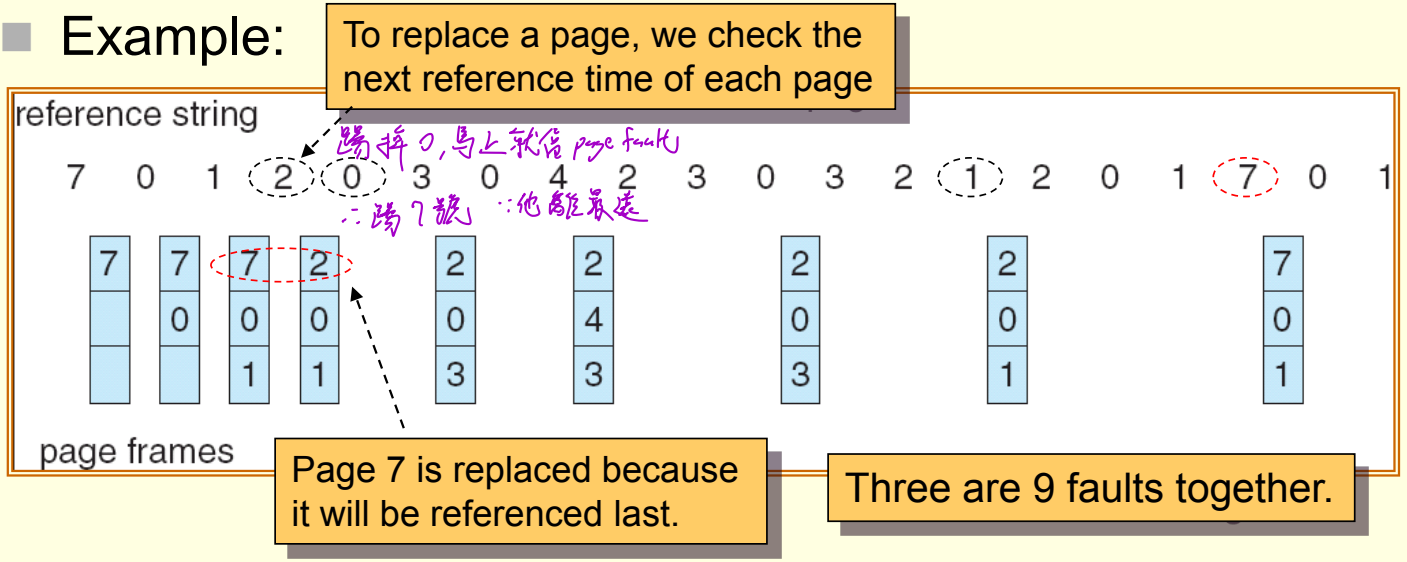
- The FIFO page-replacement algorithm is easy to understand and program.
- However, its performance is not always good.
  - The (oldest) page replaced may contain a variable that was initialized early and is in constant use.
- ***Belady's anomaly*** – for some page-replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.
  - Consider the following reference string:

    1,2,3,4,1,2,5,1,2,3,4,5



Optimal Page Replacement (1/2)

- The optimal page-replacement algorithm (also known as OPT or MIN) replaces the page that **will not be used for the longest period of time**.
  - It guarantees the lowest possible page-fault rate for a fixed number of frames.
- Example:



Optimal Page Replacement (2/2)

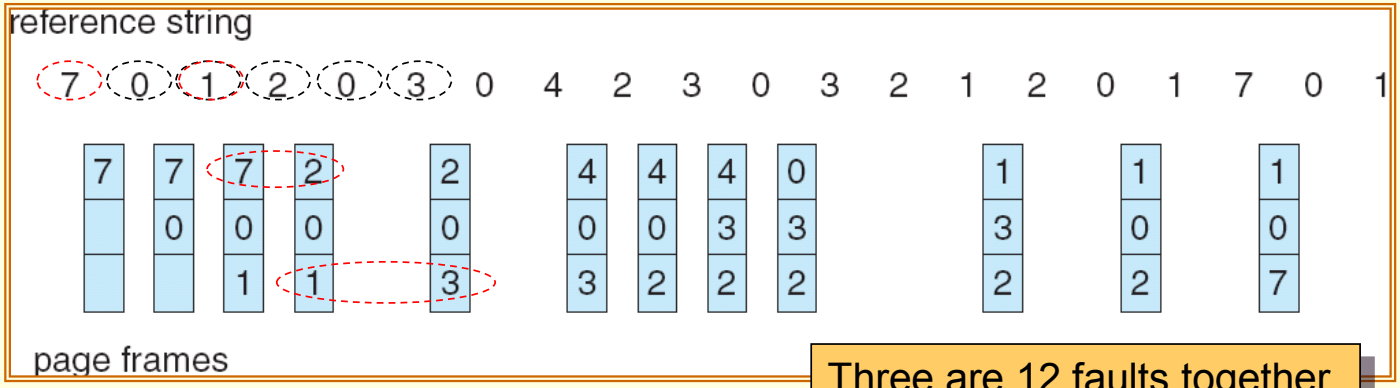
- Unfortunately, the optimal page-replacement algorithm is difficult (impossible) to implement.
  - It requires future knowledge of the reference string.
- As a result, the algorithm is used mainly for comparison studies.

LRU Page Replacement (1/4)

- ***Least-recently-used algorithm*** (LRU) - an approximation of the optimal algorithm.
- If the recent past can be used as an approximation of the near future, we can replace the page that **has not been used for the longest period of time**.
- The algorithm associates with each page the time of that page's last use.
- When a page must be replaced, LRU chooses the page that has not been used for the longest period of time.

LRU Page Replacement (2/4)

- Example:



- The LRU policy is often used as a page-replacement algorithm, **and is considered to be good**.

LRU Page Replacement (3/4)

- How to implement LRU replacement?
  - ***Counter approach***:
    - Each page-table entry has a time-of-use field.
    - CPU has a clock or counter (register). 記錄時間戳記
      - The clock is incremented for every memory reference.
    - Whenever a reference to a page is made, the contents of the clock register are copied to the time-of-use field in the page-table entry for that page.
    - In this way, we always have the time of the last reference to each page.
    - We replace the page with the smallest time value.

LRU Page Replacement (4/4)

- ***Stack approach***:
  - Whenever a page is referenced, it is removed from the stack and put on the top.
  - In this way, the least recently used page is always at the bottom of the stack.
  - To remove entries from the middle of the stack, it is best to implement this approach by using a double linked list.
    - The tail pointer points to the bottom of the stack.

- LRU and optimal replacements do not suffer from Belady's anomaly.
  - They belong to a class of page-replacement algorithms, called ***stack algorithm***.

LRU-Approximation Page Replacement (1/8)

- Implementations of LRU require hardware support.
  - **If every memory reference** calls a software to update the data structures (time-of-use field or stack), the reference will be very slow.
- However, few computer systems provide sufficient hardware support to **true** LRU replacement.
- Many systems provide help in the form of a ***reference bit***.
  - Which is the basis for many page-replacement algorithms that **approximate LRU replacement**.

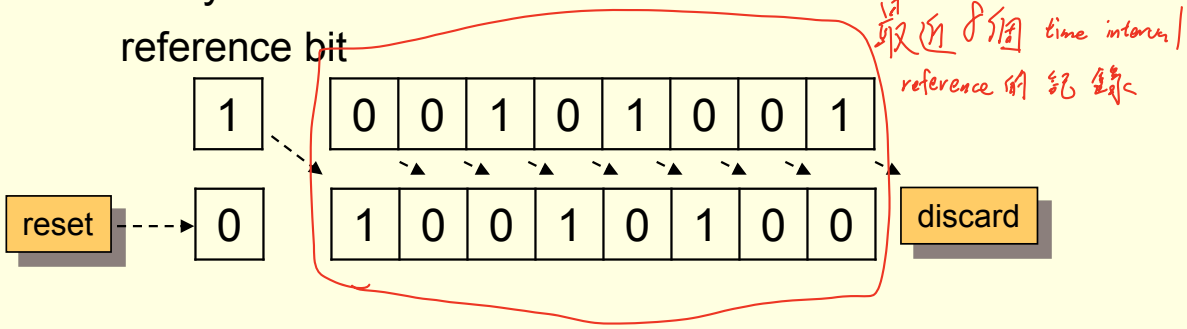
OPT
↑ approximation
LRU (counter, stack)
software solution

LRU-Approximation Page Replacement (2/8)

- Each entry in the page table has a reference bit.
- Initially, all bits are cleared.
- The reference bit is set whenever that page is referenced.
- After some time, we can determine which pages have been used by examining the reference bits.
  - Although we do not know the *order of use*.

LRU-Approximation Page Replacement (3/8)

- ***Additional-reference-bits Algorithm***:
  - We can keep an 8-bit byte for each page in a table in memory.
  - At regular intervals (e.g., 100 milliseconds), the operating system shifts the reference bit for each page into the high-order bit of its 8-bit byte.



  - The 8-bit byte thus contains the history of page use for the least eight time periods.

LRU-Approximation Page Replacement (4/8)

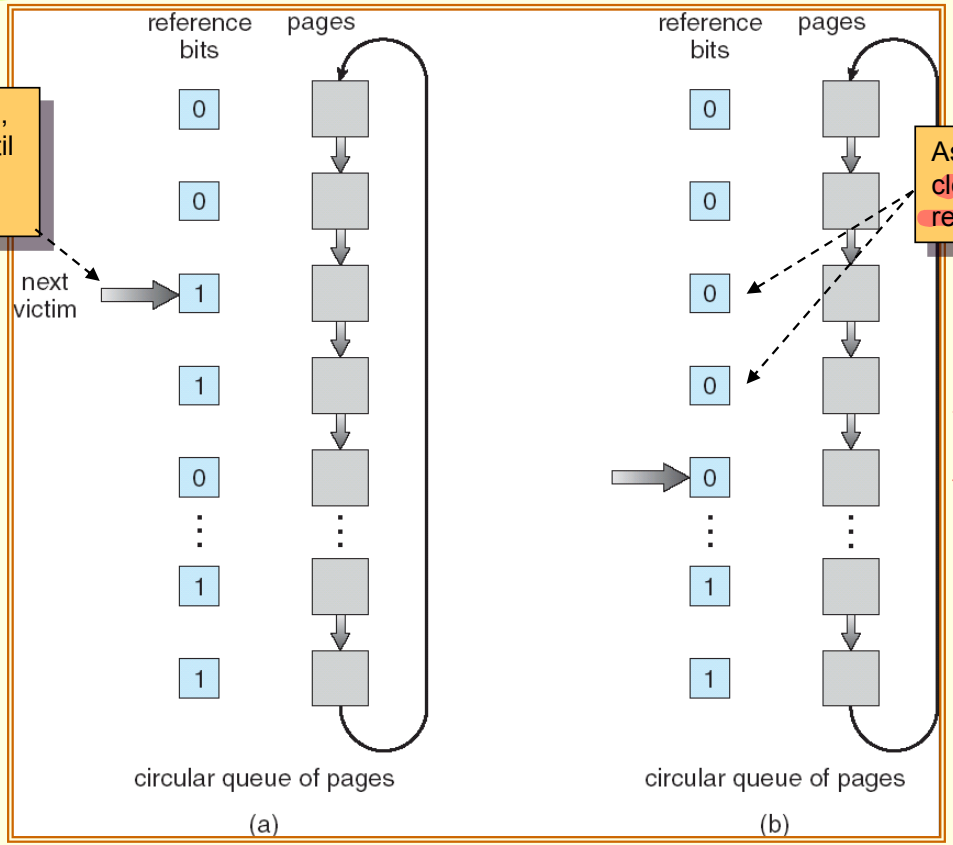
- If we interpret these 8-bit bytes as **unsigned integers**, the page with **the lowest number** is the LRU page.
- When there are more than one LRU page, we can either
  - Replace all pages with the smallest value
  - Use the FIFO method to choose among them.

LRU-Approximation Page Replacement (5/8)

- ***Second-chance algorithm***: 先有1條命
  - Sometimes referred to as the ***clock algorithm***.
  - Is a variation of the FIFO replacement algorithm.
  - When a page has been selected, we inspect its reference bit.
  - If the value is 0, we proceed to replace this page.
  - If the bit is set to 1, we give the page a second chance.
    - And move on to select the next FIFO page.
    - The reference bit of the second-chance page is cleared.
    - And its arrival time is reset to the current time.
      - That is, added to the end of the FIFO queue.
  - So, if a page is used often enough, it will never be replaced!!

LRU-Approximation Page Replacement (6/8)

- A **circular queue implementation** of the second-chance algorithm.



LRU-Approximation Page Replacement (7/8)

- In the worst case, when all bits are set …
  - The pointer cycles through the whole queue.
  - It also clears all the reference bits.
  - Then, the algorithm is identical to FIFO replacement.

- ***Enhanced Second-Chance Algorithm***:
  - Consider one additional bit – the **modify bit**.
  - With these two bits (reference bit and modify bit), each page is one of the following four possible classes:
    1. **(0,0)** neither recently used nor modified – **best page to replace**.
    2. **(0,1)** not recently used but modified – not quite as good, because the page will need to be written out before replacement.
    3. **(1,0)** recent used but clean – probably will be used again soon.
    4. **(1,1)** recently used and modified probably will be used again soon, and the page will be need to written out to disk before it can be replaced.

LRU-Approximation Page Replacement (8/8)

- When page replacement is called for, we examine the class to which that page belongs.
- We then replace the first page encountered in the lowest nonempty class.
- The major difference to the simpler clock algorithm:
  - We give preference to those page that have been modified.
  - To reduce the number of I/Os required.

Counting-Based Page Replacement (1/2)

- We can keep a **counter** of the number of references that have been made to each page.

- ***Least frequently used algorithm*** (LFU):
  - The page with the smallest count will be replaced.
  - However, if a page is used heavily during the initial phase but is never used again … it will remain in memory even though it is no longer needed.
    - One solution is to shift the counters right by 1 bit at regular intervals.

Counting-Based Page Replacement (2/2)

- ***Most frequent used algorithm*** (MFU):
  - Assume that the page with the smallest count was probably just brought in.
  - And has yet to be used.
- Generally, MFU and LFU do not approximate OPT replacement well.

Page-Buffering Algorithms (1/3)

- Also know as free-frame-buffering algorithms.
- Usually conjunct with other replacement algorithms.
- Systems commonly keep **a pool of free frames**.
- When a page fault occurs, a victim frame is chosen (by using a replacement algorithm).
- Before the victim is written out, the desired page is read into a free frame from the pool.
  - Without waiting the victim page to be written out.
  - We can re-start the process as soon as possible.
  - The victim page is **later** written out and its frame is added to the free-frame pool.

Page-Buffering Algorithms (2/3)

- An expansion is to maintain a list of **modified** pages.
- Whenever the paging device is idle, a modified page is selected as is written to the disk.
  - Its modify bit is then reset.
- Can increase the probability that a page will be clean when it is selected for replacement.
  - No written out.

Page-Buffering Algorithms (3/3)

- Keep a pool of free frames but to remember which page **was** in each frame.
- When a page fault occurs, we first check whether the desired page is in the free-frame pool.
  - If it is, the old page can be reused directly from the pool.
    - No I/O is needed.
- In some versions of UNIX system, this method is conjunct with the second-chance algorithm.
  - To reduce the penalty incurred if the wrong victim page is selected.

Allocation of Frames (1/6)

- If we have multiple processes in memory, we must decide how many frames to allocate to each process.

- The simplest case – the single-user system:
  - A system with 128 frames.
  - The operating system may take 35 frames.
  - Leaving 93 frames for the user process.

  - Initially, all 93 frames would be put on the free-frame list.
  - The first 93 page faults would all get free frames from the free-frame list.
  - When the free-frame list was exhausted, a page-replacement would be functioned.
  - When the process terminated, the 93 frames would be placed on the free-frame list.

Allocation of Frames (2/6)

- There are many variations on the simple allocation strategy.
- Many strategies require the knowledge of **minimum number of frames**.
  - The number of frames that a single instruction can reference.
  - This minimum number **is defined by the computer architecture**.
  - For example – a system in which all memory-reference instructions have only one memory address.
    - We need one frame for the instruction and one frame for the memory reference.
    - Moreover, if one-level indirect addressing is allowed (e.g., pointers), then paging requires at least three frames per process.
    - What might happen if a process had only two frames?

Allocation of Frames – **Algorithms** (3/6)

- ***Equal allocation scheme***:
  - The easiest way to split *m* frames among *n* processes is to give everyone an equal share, *m* / *n* frames.
  - For example – there are 93 frames and five processes.
    - Each process will get 18 frames.
    - The leftover three frames can be used as a free-frame buffer pool.
  - However … various processes will need differing amount of memory.
    - If a small student process only requires 2 frames, the other 16 frames are wasted!!

Allocation of Frames – Algorithms (4/6)

- ***Proportional allocation scheme***:
  - We allocate available memory to each process according to its size.

$$S = \sum s_i$$

The size of virtual memory for process p_i.

$$a_i = (s_i / S) \times m$$

The frames to p_i

The total number of frames

  - We must adjust each a_i to be an integer that is greater than the minimum number of frames required by the instruction set.

Allocation of Frames – Algorithms (5/6)

- In both schemes, the allocation may vary according to the multiprogramming level.
  - The frames that were allocated to the departed process can be spread over the remaining process.
- The **priority of process** can be a factor of frame allocation.
  - We may give the high-priority process more memory to speed its execution.
  - One solution is to use a proportional allocation scheme wherein the ratio of frames depends on the a combination of size and priority.

Allocation of Frames – **Global vs Local Replacement** (6/6)

- Frame allocation is related to the way of page replacement.
- We can classify page-replacement algorithm into two categories:
  - ***Global replacement***:
    - Allow a process to select a replacement frame from the frames currently allocated to some other process.
    - For example – high-priority processes to select from low-priority processes for replacement.
    - The number of frames allocated to a process can change.
    - More adaptive, thus generally result in greater system performance.
  - ***Local replacement***:
    - Each process selects from only its own set of allocated frame for replacement.
    - The number of frames allocated to a process does not change.

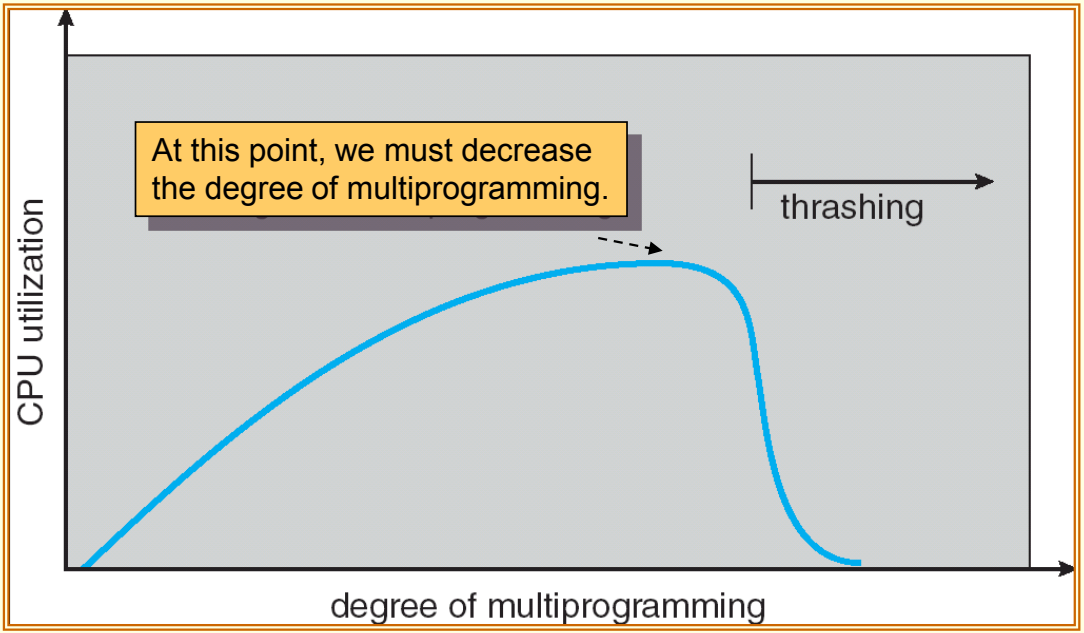
Thrashing (1/12)

- ***Thrashing*** – a process is spending more time paging than executing.
- For example:
  - If a process does not have the number of frames it needs, it will quickly page-fault.
  - It must replace some page. however … **all its pages are in active use**.
  - The replaced page will be needed again right away.
  - Consequently, it quickly **faults again, and again, and again …**

Thrashing (2/12)

- Another thrashing example – a global page-replacement system:
  - Suppose that a process enters a new phase in its execution and needs more frames.
  - It starts faulting and taking frames away from other processes.
  - However, these processes need those page. So, they also fault and take frames from other processes.
  - As a result, the paging device (disk) will queue a lot of paging requests.
  - And the ready queue would be empty.
  - The system sees the decreasing CPU utilization and increase the degree of multiprogramming.
  - Not enough memory for the new processes will cause more page faults and a longer queue for the paging device.
  - **CPU utilization drops even further!!!!!**
    - **And the system tries to increase the degree of multiprogramming even more …**

Thrashing (3/12)



Thrashing (4/12)

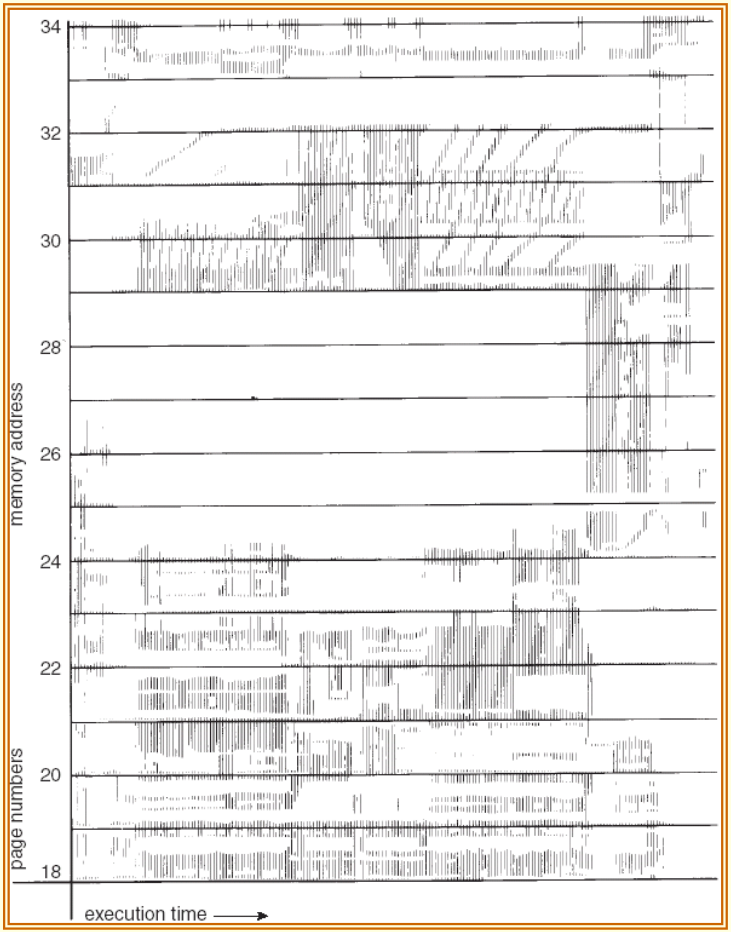
- We can **limit the effects of thrashing** by using a local replacement algorithm.
  - If one process starts thrashing, it cannot steal frames from another process can cause the later to thrash as well.
  - However … the queue of the paging device will contain a lot of requests (from the thrashing process**es**).
  - The service time for a page fault of a non-thrashing process will also increase.

- To **prevent thrashing**, we must provide a process with as many frames as it needs.

Thrashing (5/12)

- But …how do we know how many frames a process needs?
- Process's memory references generally exhibit ***locality*** patterns, or ***locality model***.
- A **locality** is a set of pages that are actively used together.
  - For example, the locality of a function call consists of the memory references of the function instructions, its local variables, and a subset of the global variables.
- The **locality model** states that, as a process executes, it moves from locality to locality.

Thrashing (6/12)

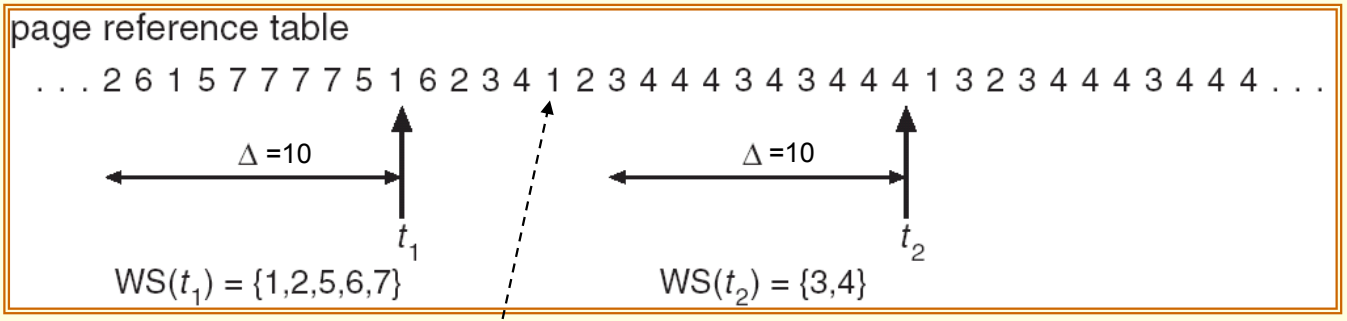


Thrashing – **Working-Set Model** (7/12)

- If we allocate fewer frames than the size of the current locality …
  - The process will thrash!!
- Based on the locality model, the ***working-set strategy*** tries to identify *how many frames a process is actually using*.
- The method uses a parameter, Δ, to define the ***working-set window***.
  - The idea is to examine **the most recent page references**.
  - The set of page in the most recent Δ page references is the ***working set***.

Thrashing – Working-Set Model (8/12)

- Examples of working set:



  - If a page (e.g., page 1) is no longer being used, it will drop from the working set.
  - Thus, the working set is an approximation of the program's locality!!

Thrashing – Working-Set Model (9/12)

- The working-set model:
  - Select the value of Δ.
  - Monitor the working set of each process and allocate to that working set enough frames.
  - If there are enough extra frames, another process can be initiated.
  - If the sum of the working-set sizes exceeds the total number of frames … that is,

The size of working set *i*

$$D = \sum WSS_i \text{ and } D > m$$

The total number of frames

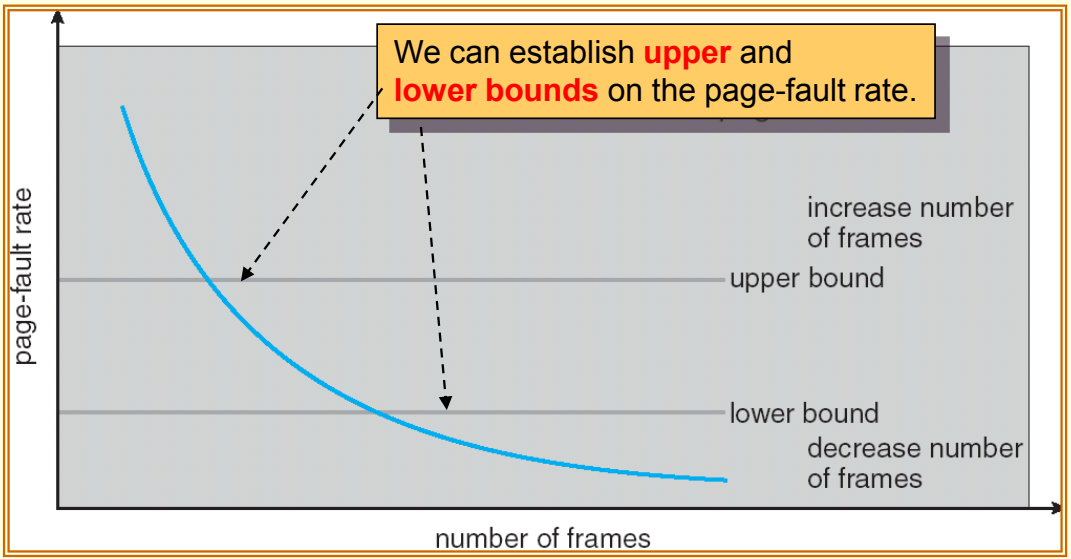
    - The operating system selects a process to suspend.
    - The process's pages are swapped and reallocated to other processes.

Thrashing – Working-Set Model (10/12)

- The working-set model prevents thrashing while keeping the degree of multiprogramming as high as possible.
  - **It thus optimizes CPU utilization!!**
- The difficulties of the model:
  - Accurately keep track the current working set.
    - The selection of Δ.
      - Too large → overlap several localities.
      - Too small → can not encompass the entire locality.

Thrashing – **Page-Fault Frequency** (11/12)

- A thrashing process has a high page-fault rate.
- To prevent it, we want to control the page-fault rate.
  - When it is too high, we know that the process needs more frames.
  - If the rate is too low, then the process may have too many frames.



Thrashing – Page-Fault Frequency (12/12)

- As with the working-set strategy, we may have to **suspend** a process.
  - When the page-fault rate of the process increases and no free frames are available.
  - The freed frames from the suspended process are then distributed to processes with high page-fault rates.

Memory-Mapped Files (1/4)

- File accesses require system call (`open()`, `read()`, and `write()`) and disk operations.
- The mechanism of ***memory-mapped files*** allow a port of the virtual address space to be logically associated with a file.
- It maps a disk block to a page (or pages) in memory.
- Initial access to the file would result in a page fault.
  - A page-sized portion of the file is read into a physical page.
- Subsequent reads and writes to the file are handled as memory accesses.
  - Thus can decrease the overhead of using the `read()` and `write()` system calls.

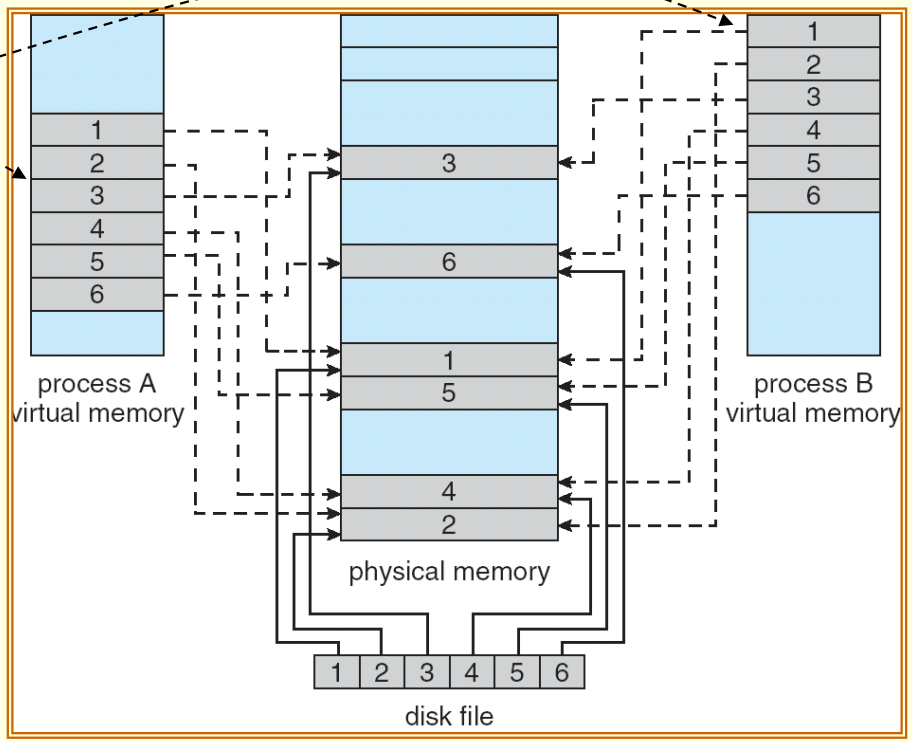
Memory-Mapped Files (2/4)

- Writes to the file mapped in memory are not necessarily immediate writes to the file on disk.
  - The operating system periodically checks whether the page in memory has been modified to update physical file.
- When the file is closed …
  - All the memory-mapped data are written back to disk.
  - And removed from the virtual memory of the process.
- Some operating systems provide memory mapping only through a specific system call.
- Some systems, e.g., Solaris, treat all file I/O as memory-mapped, allowing file access to take place via the efficient memory subsystem.

Memory-Mapped Files (3/4)

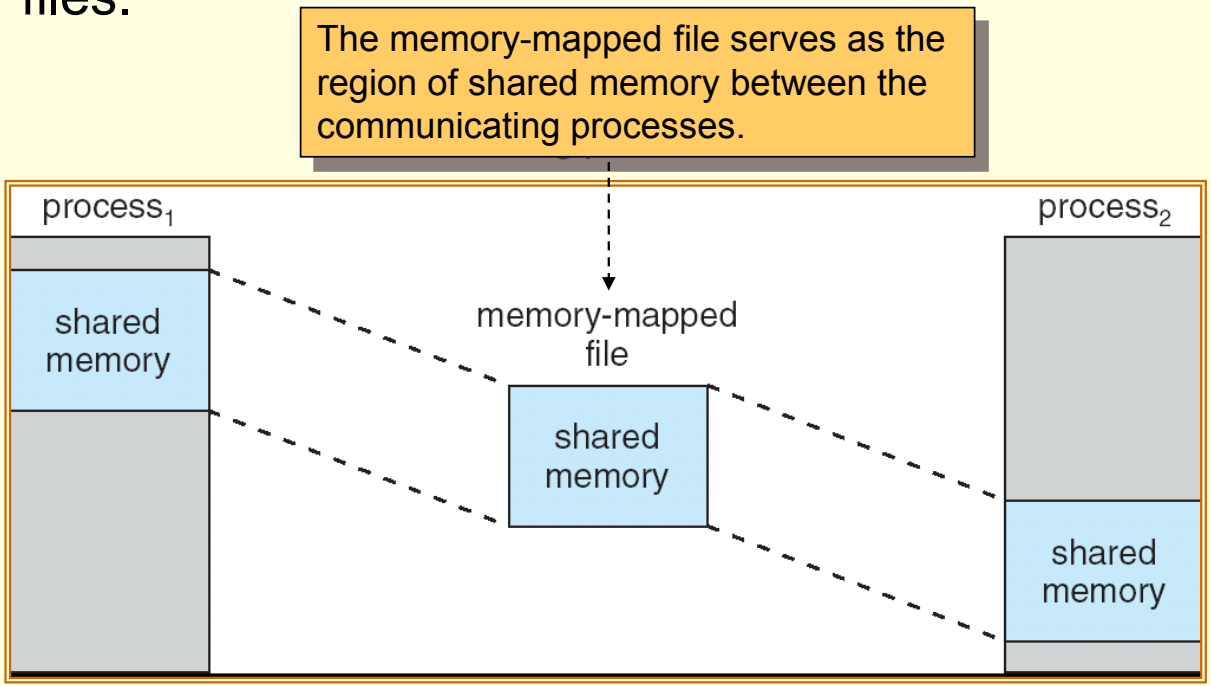
- Multiple processes may be allowed to map the same file concurrently.

Each sharing process points to the same page of physical memory – the page that holds a copy of the disk block.



Memory-Mapped Files (4/4)

- On Windows NT, 2000, and XP systems, shared memory (IPC) is accomplished by memory mapping files.



Shared Memory in the Windows API (1/8)

- To establish a memory-mapped file, a process **(producer)** first opens the file to be mapped with the `CreateFile()` function.

Producer

```
#include <windows.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
  HANDLE hFile, hMapFile;
  LPVOID mapAddress;

  // first create/open the file
  hFile = CreateFile("temp.txt",
    GENERIC_READ | GENERIC_WRITE,
    0,
    NULL,
    OPEN_ALWAYS,
    FILE_ATTRIBUTE_NORMAL,
    NULL);

  if (hFile == INVALID_HANDLE_VALUE) {
    fprintf(stderr,"Could not open file temp.txt (%d).\n",GetLastError());
    return -1;
  }
```

file name

read/write access

open new or existing file

no sharing

default security

Shared Memory in the Windows API (2/8)

- Producer then creates a mapping of this file (`HANDLE`) using the `CreateFileMapping()` function.

```
// now obtain a mapping for it

hMapFile = CreateFileMapping(hFile,
  NULL,
  PAGE_READWRITE,
  0,
  0,
  TEXT("SharedObject"));

if (hMapFile == NULL) {
  fprintf(stderr,"Could not create mapping (%d).\n", GetLastError());
  return -1;
}
```

file handle

default security

read/write access to mapped pages

map entire file

named shared memory object, used by other communicating processes

Shared Memory in the Windows API (3/8)

- The process then establishes a view of the mapped file in its virtual address space with the `MapViewOfFile()` function.

```
// now establish a mapped viewing of the file

mapAddress = MapViewOfFile(hMapFile,
  FILE_MAP_ALL_ACCESS,
  0,
  0,
  0);

if(mapAddress == NULL) {
  printf("Could not map view of file (%d).\n", GetLastError());
  return -1;
}
```

mapped object handle

read/write access

mapped view of entire file

Return a pointer to the starting address of the mapped view

Shared Memory in the Windows API (4/8)

- The producer process writes the message "`Shared memory message`" to shared memory.

```
// write to shared memory

sprintf(mapAddress,"%s","Shared memory message");
```

- Finally, the process removes the view of the mapped file and the handles.

```
// remove the file mapping
  UnmapViewOfFile(mapAddress);

  // close all handles
  CloseHandle(hMapFile);
  CloseHandle(hFile);
}
```

Shared Memory in the Windows API (5/8)

- The **consumer** process first creates (opens) a mapping to the existing named shared-memory object.

Consumer

```
#include <stdio.h>
#include <windows.h>

int main(int argc, char *argv[]) {
  HANDLE hMapFile;
  LPVOID lpMapAddress;

  hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS,
    FALSE,
    TEXT("SharedObject"));

  if (hMapFile == NULL) {
    printf("Could not open file mapping object (%d).\n",
       GetLastError());
    return -1;
  }
```

R/W access

no inheritance

name of mapped file object

Shared Memory in the Windows API (6/8)

- The consumer process must also create a view of the mapped file.

```
lpMapAddress = MapViewOfFile(hMapFile,
  FILE_MAP_ALL_ACCESS,
  0,
  0,
  0);

if (lpMapAddress == NULL) {
  printf("Could not map view of file (%d).\n", GetLastError());
  return -1;
}
```

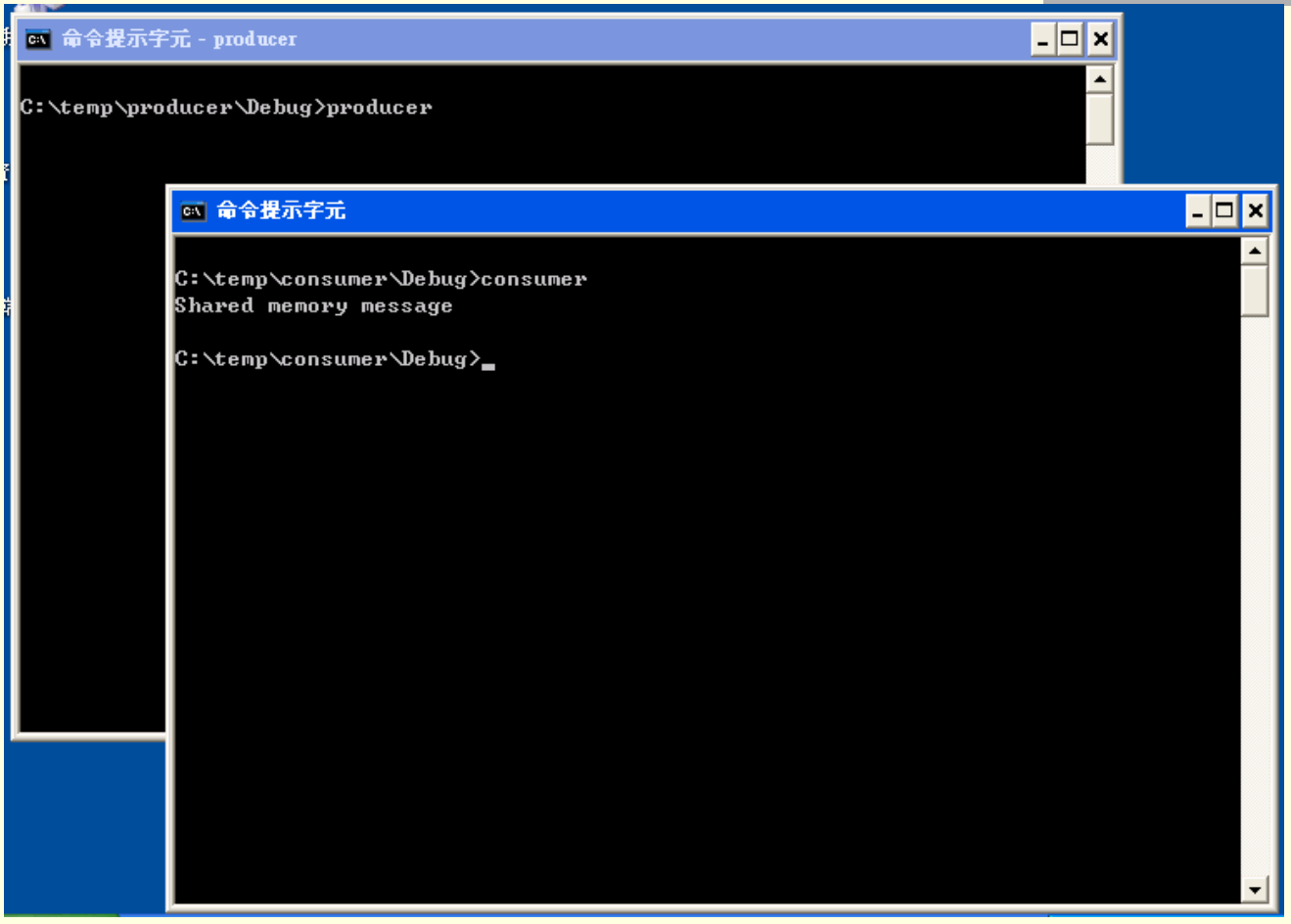
Shared Memory in the Windows API (7/8)

- The process then reads from shared memory the message "`Shared memory message`" that was written by the producer process.
- Finally, close the mapping and handle.

```
  printf("%s\n",lpMapAddress);

  UnmapViewOfFile(lpMapAddress);
  CloseHandle(hMapFile);
}
```

Shared Memory in the Windows API (8/8)



Other Issues and Considerations

- Allocating kernel memory.
- Pre-paging.
- Page size.
- …

End of Chapter 9